Day 88: Subword Embeddings and Out-of-Vocabulary Words (fastText From Scratch)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 5 | Complete study guide

Watch: FastText: Solving the Out-of-Vocabulary Problem with Subword Embedding (Dr. Partha Majumdar) | FastText Explained: Subword Embeddings in NLP (Tech Pulse Labs)

Script: `day88_subword_embeddings.py` · every number in this guide is copied by program from `day88_run_output.txt`, the real output of that script.

Word2Vec has nothing for a word it never saw. fastText builds every word's vector from its character pieces, so any string gets one. Tested honestly, on 31 words removed from the corpus before training, the result is mixed: fastText's vectors for them find a same-group word 0.290 of the time, far above chance (0.082) but below plain spelling similarity (0.355). The reason is measured, not guessed: pieces shared by many words grow long and decide every vector, so on two million words fastText mostly hears a word's common endings.

0. Where today fits

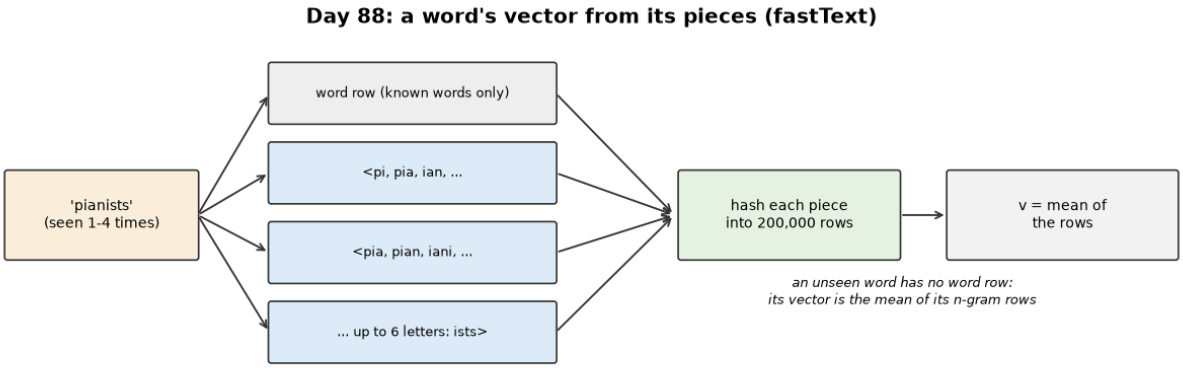
DEFINITION — start here, no prior knowledge assumed

A word is **out of vocabulary** (OOV) when the model has no vector for it, because it never appeared in training or appeared too rarely to be kept. **Subword embeddings** give vectors to pieces of words instead. fastText cuts each word, wrapped in < and >, into every **character n-gram** of 3 to 6 letters, keeps one row per piece (pieces are hashed into a fixed number of rows), and makes the word's vector the mean of its pieces' rows plus a row for the word itself if it is known.

REAL-WORLD SCENARIO

Meeting the word 'unhelpfulness' for the first time: 'un-', 'help', '-ful' and '-ness' already say most of what it means.

SYSTEM DIAGRAM



A word's vector from its pieces.

Today's part for Day 90

`day88_exercise.py` is the subword part on its own: the n-grams, the fixed hash, and the mean of rows with its gradient. Day 90 uses `char_ngrams` to ask whether the words its model invents are built from real pieces.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. A word as its pieces

DEFINITION — start here, no prior knowledge assumed

The n-grams of 'where', and how all the vocabulary's n-grams fit into 200,000 rows. Hashing is a function from any string to a row number; two different pieces can land in the same row (a collision).

REAL-WORLD SCENARIO

Filing papers by the last three digits of a number: any number has a drawer, and some drawers are shared.

MATHEMATICS

$$G(w) = \text{n-grams of '<' + } w \text{ + '>', lengths 3 to 6;} \quad row(g) = V + FNV\text{-}1a(g) \bmod 200{,}000$$

$$v_w = \text{mean of } W_{in}[w] \text{ and } W_{in}[row(g)] \text{ for } g \text{ in } G(w); \quad \text{unseen word: mean over } G(w) \text{ only}$$

IMPLEMENTATION

```
def char_ngrams(word, nmin=NMIN, nmax=NMAX):
    w = "<" + word + ">"
    return [w[i:i + n] for n in range(nmin, nmax + 1) for i in range(len(w) - n + 1)]
def fnv1a(s):
    h = 2166136261
    for b in s.encode("utf-8"):
        h = ((h ^ b) * 16777619) & 0xFFFFFFFF
    return h
def ngram_rows(word, V):
    return [V + fnv1a(g) % BUCKETS for g in char_ngrams(word)]
```

ACTUAL OUTPUT

```
'where': 14 n-grams: <wh whe her ere re> <whe wher here ere> <wher where here> <where where>
the piece '<her>' (the whole word 'her') is among the n-grams of 'her': True; of 'where': False ('where' has
    'her' but not '<her>')
the 21650 known words have 125,097 distinct n-grams; they fill 93,157 of the 200,000 rows (31,940 n-grams
    share a row with another); a word has 23.1 n-grams on average
Actual output from this lesson's run.
```

What it means

A word has 23.1 pieces on average. The 125,097 distinct pieces of the vocabulary land in 93,157 rows, so 31,940 share a row with another piece: collisions are common and are part of the method. The end marks matter: '<her>' (the word 'her') is a different piece from the 'her' inside 'where'.

WHY CHOOSE THIS?

Any string of letters gets a vector; memory is fixed by the number of rows.

WHY NOT THIS? (TRADEOFFS)

Collisions mix unrelated pieces; a word has about 24 rows to average.

2. Checking the subword input

DEFINITION — start here, no prior knowledge assumed

The loss is Day 86's negative-sampling loss; only the centre vector changes: it is now the mean of the word's rows, so each row gets 1 / (number of rows) of the word's gradient.

REAL-WORLD SCENARIO

A team result shared equally among the members who produced it.

MATHEMATICS

$dW_{in}[g] \mathrel{+}= dv_w / |G(w)|$ *for every row g of word w (rows used by several words add up)*

IMPLEMENTATION

```
def step(self, c, tgt, lr):
    rows, w, seg, n = self.gather(c)
    v = np.add.reduceat(self.Win[rows] * w[:, None], seg, axis=0)
    U = self.Wout[tgt]
    z = np.einsum("nd,nkd->nk", v, U)
    loss = np.logaddexp(0, -z[:, 0]).sum() + np.logaddexp(0, z[:, 1:]).sum()
    gz = d86.sigmoid(z); gz[:, 0] -= 1
    dv = np.einsum("nk,nkd->nd", gz, U)
    np.add.at(self.Win, rows, -lr * w[:, None] * np.repeat(dv, n, axis=0))
    np.add.at(self.Wout, tgt, -lr * gz[:, :, None] * v[:, None, :])
    return loss
```

ACTUAL OUTPUT

```
gradient check: 92 entries (3 word rows + 20 buckets, 4 numbers each), largest |numeric -
    analytic| = 7.37e-10, all pass: True
Actual output from this lesson's run.
```

What it means

Every entry passes (7.37e-10) with only 20 rows, so pieces of 'cat', 'cats' and 'dog' collide as they would in the real table. The check runs through step() itself, the code that trains.

WHY CHOOSE THIS?

The training code is the checked code.

WHY NOT THIS? (TRADEOFFS)

A tiny table; the real one has 200,000 rows.

3. Two models, one corpus: the known words

DEFINITION — start here, no prior knowledge assumed

31 words (every second word of six of Day 86's groups) are removed from the corpus. Then Word2Vec and fastText are trained on what is left with Day 86's settings, and Day 86's scores are measured after every epoch on the 59 category words both models know.

REAL-WORLD SCENARIO

Two students take the same course; afterwards both are examined on the same known material.

SPECIFICATION

Held out	two four six eight ten; january march june august october december; red green black brown purple; hydrogen carbon iron silver helium; piano violin flute accordion synthesizer; physics biology astronomy economics philosophy.
Both models	d = 50, window 5, 5 noise words, subsampling, 5 epochs, seed 0; learning rate 0.025 (Word2Vec), 0.05 (fastText's default).
fastText	n-grams of 3 to 6 characters, 200,000 rows.

Why this test

Subwords should not cost quality on the words a model does know.

ACTUAL OUTPUT

```
model      epoch  category  capitals  gender  s/epoch
Word2Vec       1     0.797     0.029   0.056       67
Word2Vec       2     0.847     0.048   0.144       64
Word2Vec       3     0.881     0.096   0.178       61
Word2Vec       4     0.915     0.129   0.233       71
Word2Vec       5     0.932     0.151   0.267       71
fastText       1     0.559     0.004   0.056      280
fastText       2     0.847     0.007   0.078      276
fastText       3     0.932     0.004   0.156      281
fastText       4     0.932     0.011   0.156      279
fastText       5     0.932     0.011   0.167      278
Actual output from this lesson's run.
```

What it means

Both reach 0.932 on the category check, fastText from a slower start (0.559 after one epoch). The analogies are another matter: fastText ends at 0.011 (capitals) and 0.167 (gender), Word2Vec at 0.151 and 0.267. Section 6 suggests why: a fastText vector is dominated by pieces shared with many other words, which blurs the differences analogies rely on.

WHY CHOOSE THIS?

Same word groups as Word2Vec.

WHY NOT THIS? (TRADEOFFS)

Much weaker analogies at this corpus size.

4. Words never seen

DEFINITION — start here, no prior knowledge assumed

For each held-out word, fastText builds a vector from its n-grams; which known category word is most similar? Right if it is in the same group ('january' → 'february'). Two baselines: **spelling only** (the known word sharing the largest share of n-grams, the Jaccard similarity) and **chance** (a random known category word).

REAL-WORLD SCENARIO

Guessing the meaning of a new word from its spelling, against guessing from experience of its pieces.

MATHEMATICS

$$Jaccard(a, b) = |G(a) \cap G(b)| \,/\, |G(a) \cup G(b)|$$

ACTUAL OUTPUT

```
method                   right
Word2Vec                   no vector: never saw the words
fastText, n-grams        0.290
spelling only            0.355
chance (random pick)     0.082
per group, fastText / spelling:
  number      5 words   0.40 / 0.40
  month       6 words   0.17 / 0.50
  colour      5 words   0.20 / 0.20
  element     5 words   0.20 / 0.40
  instrument  5 words   0.00 / 0.00
  science     5 words   0.80 / 0.60
nearest known words of the whole vocabulary (fastText):
  january    -> pbs, mcgraw, gwynedd, ludovico, sergio, lulu
  four       -> honour, odour, timur, vigour, ashur, fourth
  physics    -> chemistry, physiological, pataphysics, robotics, biochemistry, metaphysics
  green      -> greenwich, greene, greens, tlemcen, greenhouse, mosque
  silver     -> enver, closer, quaker, keeper, viper, gallbladder
  violin     -> zulu, zeppelin, quincy, btr, mcdivitt, griffin
Actual output from this lesson's run.
```

What it means

Word2Vec cannot answer at all. fastText's vectors carry real information (0.290 against 0.082 by chance), and for sciences they beat spelling (0.80 / 0.60). Overall they lose to spelling alone (0.355). The neighbours show why: 'four' sits with 'honour', 'odour' and 'vigour' (the ending '-our>'), 'silver' with 'closer' and 'keeper' ('-er>'). 'physics' works because '-ics' words are sciences. Only 31 test words: one word moves a group's score by 0.17 to 0.20.

WHY CHOOSE THIS?

A vector for every word, and better than chance.

WHY NOT THIS? (TRADEOFFS)

On two million words, not better than spelling; endings dominate.

5. Rare real words

DEFINITION — start here, no prior knowledge assumed

Words seen only 1 to 4 times are dropped from both vocabularies. 400 of them made of a known stem plus a common ending ('pianists' = 'pianist' + 's') are tested: is the stem among the word's 10 most similar known words?

REAL-WORLD SCENARIO

Recognising 'griefs' as a form of 'grief'.

ACTUAL OUTPUT

```
56701 rare words; 400 sampled with a known stem
stem among the 10 most similar known words: fastText 0.182, spelling only 0.993
examples (fastText's 4 most similar known words):
  aargauer          -> bodybuilder, grimm, westminster, squealer
  bronzes           -> diomedes, hellenes, dardanelles, miracles
  digitally         -> graphically, adaptable, inevitable, unsuitable
  griefs            -> miraculous, wessex, grief, miraculously
  lettered          -> corrupted, embraced, represented, inscribed
  physiques         -> physique, psychiatric, phys, phylogeny
  samoans           -> cubans, koreans, bangladesh, spaniards
  tattooing         -> loosening, kidnapping, crying, judging
Actual output from this lesson's run.
```

What it means

fastText finds the stem 0.182 of the time; spelling finds it 0.993 of the time. This test favours spelling by design (the stem shares most of the word's n-grams), so it is a check of how much fastText keeps of a word's own pieces, and the answer is: little. Its neighbours share endings and roles instead: 'samoans' → 'cubans', 'koreans'; 'lettered' → 'inscribed', 'represented'. Sometimes that is useful information (both are plural nationalities), but it is not the stem.

WHY CHOOSE THIS?

Rare words get vectors that share their grammatical role.

WHY NOT THIS? (TRADEOFFS)

The word's own stem is mostly lost.

6. Why: which rows dominate a word's vector

DEFINITION — start here, no prior knowledge assumed

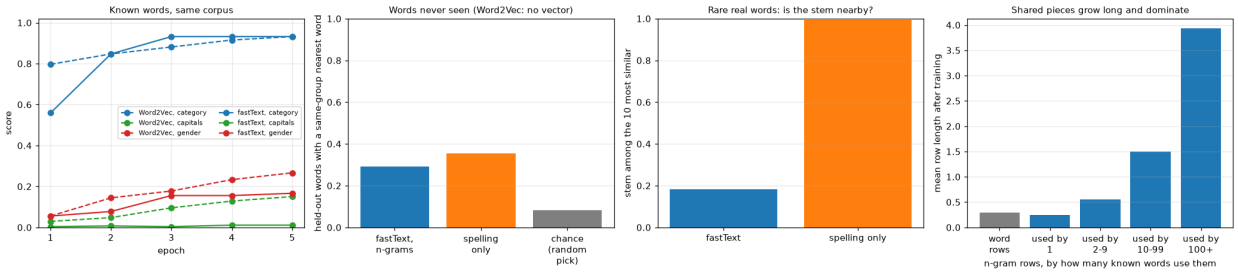
A word's vector is a mean of rows. If some rows are much longer than others, they decide its direction. Measured after training: the mean length of the rows, grouped by how many known words use each n-gram row.

REAL-WORLD SCENARIO

A committee vote where a few members have megaphones.

ACTUAL OUTPUT

```
row                              rows  mean length
word rows (one per word)        21650        0.290
n-gram, used by 1 words         38324        0.242
n-gram, used by 2-9 words       45509        0.550
n-gram, used by 10-99 words      8872        1.494
n-gram, used by 100+ words        452        3.930
known words: cosine between the n-gram-only vector and the full vector (word row included), mean of 300:
    0.9996
Actual output from this lesson's run.
```



Left: known-word scores by epoch (dashed: Word2Vec). Then the held-out words, the rare-word stem test, and row length by how many words share the row.

What it means

Rows shared by 100 or more words are 3.930 long on average, rows used by one word 0.242, and the words' own rows 0.290. Every time any word containing '-ing' trains, the '-ing' rows move, so common pieces grow; rare pieces, which carry a word's identity, hardly move. For known words the n-gram-only vector and the full vector are almost identical (cosine 0.9996): the word's own row barely counts. That is why fastText hears endings. Published fastText vectors are trained on billions of words, where rare pieces also get trained; that setting was not tested here.

WHY CHOOSE THIS?

One measurement that offers an explanation for every result above.

WHY NOT THIS? (TRADEOFFS)

One corpus size and one setting of n-gram lengths and rows.

7. What the subwords cost

ACTUAL OUTPUT

```
weights: Word2Vec 2,165,000 (two tables of 21650 x 50); fastText 12,165,000 (plus 200,000 n-gram
    rows)
time per epoch in this run: Word2Vec 67 s, fastText 279 s (4.2 times; median of 5 epochs)
Actual output from this lesson's run.
```

fastText has 12,165,000 weights against 2,165,000: the 200,000 n-gram rows are most of them. In this run an epoch took 279 s against 67 s (4.2 times), but timings depend on what else the machine is doing; every word now updates about 24 rows (its own and about 23 n-grams) instead of one, which is the dependable part of the cost.

Interview preparation — technical questions

Q: How does fastText build a word vector?

A: The mean of the rows of its character n-grams (3 to 6 letters, with < > marks) and of the word itself.

Q: Why hash the n-grams?

A: A fixed number of rows for any number of pieces; collisions are accepted.

Q: How does fastText handle an unseen word?

A: The mean of its n-gram rows; there is no word row.

Q: Why use a fixed hash such as FNV-1a?

A: Python's hash() changes between runs; the rows must be the same every time.

Q: What does an n-gram row's gradient look like?

A: Each row of a word gets the word's gradient divided by its number of rows.

Interview preparation — theoretical questions

Q: Why can subword vectors be dominated by endings?

A: Shared pieces are trained by many words and grow long; measured here, 3.930 against 0.242.

Q: Is fastText better than spelling for unseen words?

A: Not here (0.290 against 0.355); better than chance (0.082) and than nothing.

Q: Why did fastText's analogies collapse?

A: Likely because vectors dominated by shared pieces lose the word-specific differences analogies need (not tested directly).

Q: When would subwords help most?

A: Languages with rich morphology, typos, and large corpora where rare pieces are trained too.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Subwords give every word a vector, including words never seen. On two million words, those vectors beat chance but not plain spelling, because the pieces shared by many words grow long and dominate every mean: fastText learns a lot about endings and little about the rest.

- Held-out words, same-group nearest word: fastText 0.290, spelling 0.355, chance 0.082; Word2Vec: no vector.
- Rare words, stem among the 10 nearest: fastText 0.182, spelling 0.993.
- Known words: category 0.932 for both; analogies far weaker for fastText.
- Row length: shared by 100+ words 3.930, used by one word 0.242.

Suggested watch order, if starting from zero

- 1. FastText Explained: Subword Embeddings in NLP (Tech Pulse Labs) — subword embeddings and character n-grams.
- 2. FastText: Solving the Out-of-Vocabulary Problem with Subword Embedding (Dr. Partha Majumdar) — fastText's architecture and out-of-vocabulary words.

Day 89 preview

- **Pretrained vectors.** GloVe from co-occurrence counts, saved in GloVe's text format, loaded back, and used as the starting table of Day 85's tagger with plenty of labels and with few.